虚拟化开发框架 common-virtualization 介绍

1. 开发目的

1.1 提升虚拟化项目的开发效率

(1) 大幅降低虚拟化开发所需代码量

	service 代码行数	sdk 代码行数	agent 代码行数	总计
tstack(未使用框架)	11961	3546	2999	18506
tstack(使用框架)	2952	2795	-	5747
差量百分比	405.18%	126.87%	-	322.01%

表 1.1 使用框架前后代码量对比(仅统计 java 源代码，不计空行和注释)

表 1.1 中列出了对接 tstack 平台时，使用该框架和不使用该框架开发所需要的代码量，两次对接除了平台版本不同外没有其他区别。

该框架对大量公共逻辑进行了抽象封装，从而使开发人员对接新平台时仅需写少量的业务逻辑的代码即可，因此使开发工作量大幅度下降。

(2) 降低开发门槛

在本框架中，开发难度较高的底层代码都已经进行了抽象封装，业务逻辑中并不涉及复杂的计算机操作系统底层的知识，因此开发者只需要能够熟练使用 java 语言并且对所对接的虚拟化平台有充分了解就能够使用本框架进行开发。其中虚拟化平台部分的抽象封装将在该框架后续版本中进行，封装完毕后可再次降低开发者对虚拟化平台的了解程度这个开发门槛。

(3) 维护集中化

如使用本框架作为后续虚拟化开发的基础，框架中各模块将为所有项目业务逻辑可能用到指令提供接口支持，仅需要保证框架中的接口实现正确和业务逻辑的正确，就能保证业务正确。当出现非业务逻辑的错误时，仅需要针对出错模块进行一处修复就可修复该错误在多个项目中引起的同一类问题。

1.2 使虚拟化项目标准化

(1) 错误码和异常处理

错误码和异常处理也进行了模块化处理，每个模块内部处理自己的遇到的异常并抛出封装后的异常。经模块封装后的一种异常对应一个错误码，使问题反馈更清晰，更好进行定位出错的地方，同时也能降低出现未捕获的异常使程序崩溃的风险。

(2) i18n

由于对错误码进行了模块化处理，可以很方便地对错误码信息进行 i18n 映射。

1.3 提供团队协同的基础

该框架定义了开发业务逻辑使用的基类，在基类中定义了需要实现的方法。因此使用该框架后，开发时只需要在特定区域填写代码即可。项目结构一致性较强，从而保证了代码的可维护性。

该框架对涉及到操作系统底层的接口逻辑进行了高度的抽象化处理，解耦了业务逻辑对系统环境的依赖性。在框架后续版本添加对虚拟化平台的抽象后，将解耦业务逻辑对虚拟化平台的依赖性(最理想的状态)。

在解除了业务逻辑和系统交互逻辑的耦合后，业务逻辑代码可以交由普通研发人员进行开发，而系统交互模块中的代码可以交由专家级别的人员研发，将分工明确化。

同时解耦的另一个好处就是可以为业务逻辑代码提供 mock 测试(一种模拟代码运行在部署环境中的测试)，该测试可以解决目前虚拟化代码调试难度大的问题，将进一步提升研发效率，同时大幅降低逻辑错误的发生。

1.4 提供了技术纵向发展的时间和实现基础

结合以上所述，在大幅提升研发效率和降低代码管理成本后，该框架所带来最大的好处是能够让研发人员腾出时间和精力去研究和优化核心技术。

在之后项目结构介绍中可以看到，承载性能的代码都集中在 access-local 模块和 access-platform 模块，因此这两个模块将成为业务纵向发展的核心模块，为技术优化提供了抽象基础和方向。

2. 项目结构

2.1 业务交互模型

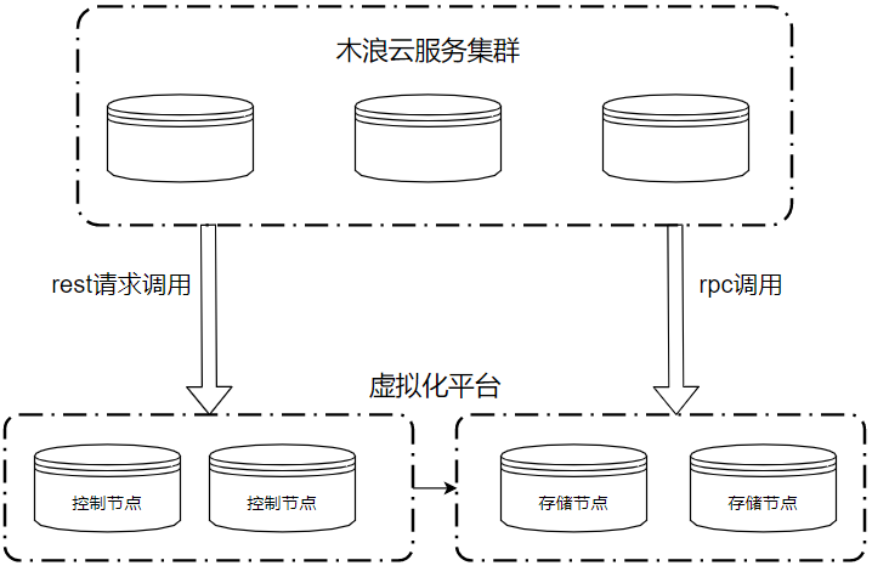

图 2.1 服务交互模型

木浪云融合数据管理平台的虚拟化备份业务需要与所对接虚拟化平台的控制节点和存储节点做交互。与控制节点的交互主要涉及快照创建、虚拟机创建、卷创建等控制虚拟化资源的指令，一般通过 restful api 请求或命令行客户端调用。与存储节点的交互主要是数据传输指令，包括挂载 nfs 作为数据接收目录、导出数据等指令，一般通过 rpc 调用控制。

2.2 业务调用栈

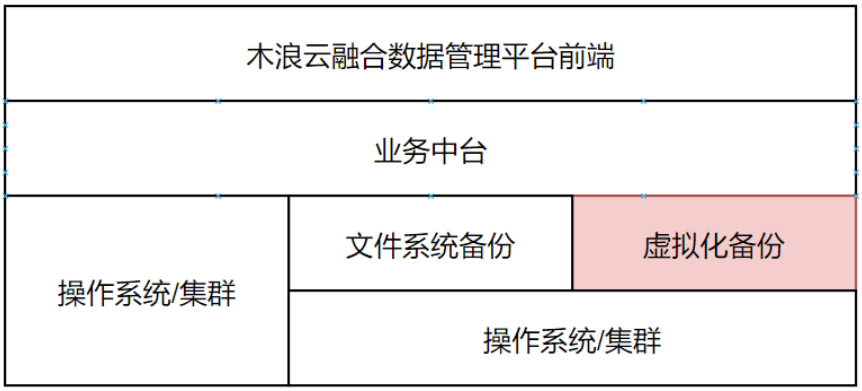

图 2.2 木浪云业务调用栈

木浪云融合数据管理平台结构大致分为三层：前端、中台、后台。虚拟化备份为后台的一部分。

2.3 虚拟化业务调用栈

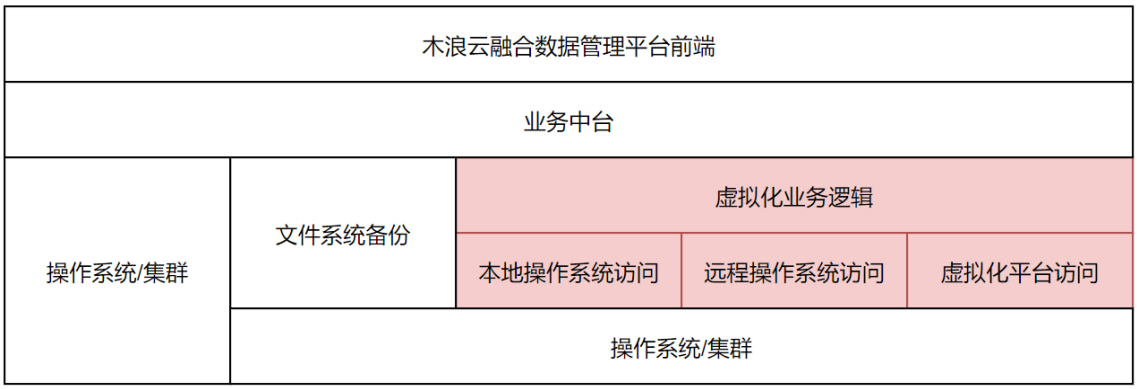

图 2.3 虚拟化备份后台细化

虚拟化备份后台的调用栈又可大致分为两层共四块逻辑调用组成，分别是上层的业务逻辑模块和下层的本地操作系统访问模块、远程操作系统访问模块、虚拟化平台访问模块。

上层业务逻辑模块作为控制模块，负责对下层提供的各种指令进行组合从而实现特定的功能。

下层三个模块作为桥梁模块，向上提供指令调用的接口，向下则调用底层操作系统/集群的指令。这层模块的作用是对业务逻辑层屏蔽操作系统/集群，使业务逻辑层不直接与操作系统/集群进行交互。这样做的好处是能够解耦业务逻辑对环境的依赖。

2.4 虚拟化各功能模块介绍

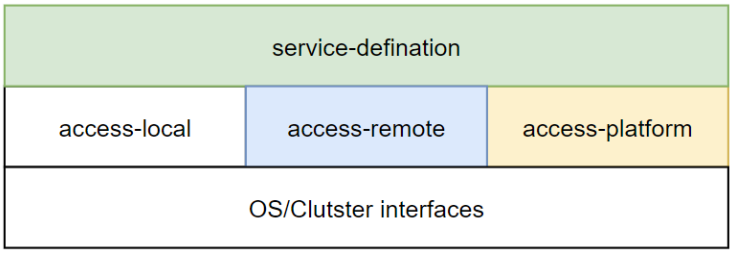

图 2.4 虚拟化备份框架中模块调用栈

图 2.4 上两层是图 2.3 中各个模块对应的 Maven module。其中 service-defination 对应业务逻辑模块，access-local 对应本地操作系统访问模块，access-remote 对应远程操作系统访问模块，access-platform 对应虚拟化平台访问模块。

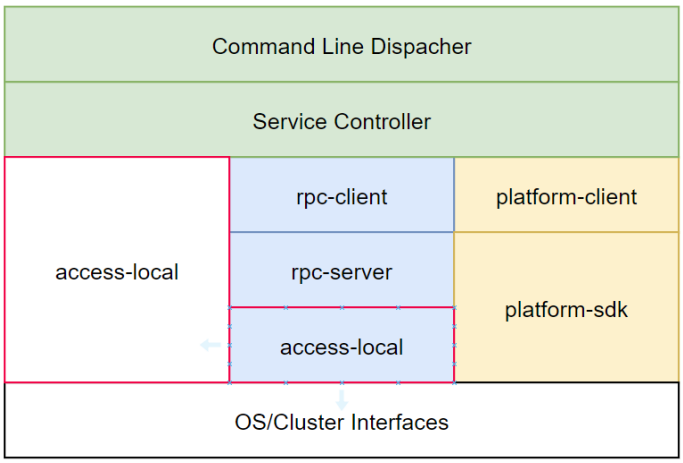

图 2.5 虚拟化业务各模块详细调用栈

(1) service-defination 模块

该模块分为两层: 第一层为 Command Line Dispacher，该层负责接收命令行传递参数，解析后传入到下层。第二层 Service Controller，该层接收上层解析后传递的参数对象，执行对应的业务逻辑。

(2) access-local 模块

该模块负责解除上层业务逻辑对操作系统环境的耦合，向上提供调用操作系统底层指令的接口，向下直接与操作系统进行交互。

(3) access-remote 模块

该模块调用栈共三层: 第一层为 rpc-client 模块，该层的作用是对上层屏蔽 rpc 调用过程中网络通信的过程，解耦 rpc 调用对服务端环境的依赖，向上提供 rpc 调用的接口，向下发送 http/https 请求至远程服务器调用 rpc 命令；第二层为 rpc-server 模块，该层作为 rpc 调用的服务端，向上接收 http/https 请求，向下调用第三层 access-local 模块提供的操作系统指令。

(4) access-platform 模块

该模块调用栈共两层: 第一层为 platform-client，该层的作用是解除业务逻辑对虚拟化集群的耦合，向上提供调用虚拟化备份业务所需要的指令，如创建指定大小的

虚拟机、创建指定大小的磁盘等，向下调用平台 sdk 执行具体操作。第二层为 platform-sdk 层，该层一般由所对接的虚拟化平台官方提供，作用是抽象与虚拟化平台集群的交互，向上提供与虚拟化平台的交互接口，向下一般通过 http/https 请求向虚拟化平台所处集群发送指令。该模块抽象难度较大，现版本暂未完成，在后续版本中会进行实现。

2.5 框架的性能承载模块

在图 2.5 中可以发现，该框架通过 access-local 模块完全解耦了上层逻辑与操作系统的交互，通过 platform-access 模块完全解耦了上层逻辑与虚拟化集群的交互。因此，这两个模块将作为服务性能的承载模块。

3. 该框架未来版本规划

3.1 对虚拟化平台的交互操作进行抽象封装(构建 access-platfrom 模块)

该计划的目的是完全解耦业务逻辑代码对环境的依赖性，为开发 mock 环境提供基础。

3.2 基于 access-local 模块和 access-platform 模块构建 mock 环境

mock 环境是该框架下一个版本的开发重点。该环境可大大降低开发业务逻辑时的调试难度，从而提升业务逻辑代码的开发效率。若 mock 环境足够完善，可以避免各种由环境问题(如虚拟化平台测试环境搭建、本地测试环境搭建)所带来的开发堵塞。同时在整体研发流程上可以构建一套针对业务逻辑漏洞的测试，缩短项目测试周期。

3.3 固化已有项目

将目前已经对接过的虚拟化项目迁移为使用该框架开发的项目，这样做的好处是：

(1) 可以将已经出现但未解决的问题集中解决，同时将业务核心技术向深层次推进。

(2) 降低各个项目的管理成本和维护成本。

(3) 持续完善框架，使框架普适性更强。

3.4 优化 access-local 模块与 access-platform 模块

access-local 模块直接与操作系统交互，优化空间较大，甚至基于不同的操作系统可以有不同的实现逻辑，优化工作主要考量优化人员对操作系统的熟悉程度。

access-platform 模块直接与虚拟化集群交互，不同的虚拟化平台有不同的实现逻辑，优化工作主要考量优化人员对于虚拟化平台的熟悉程度。

3.5 项目框架服务化改造

将虚拟化项目由命令行调用改为服务化调用，同时添加新的桥接入口类提供 cli 转 restful 请求的通道，以兼容现阶段的命令行调用。

3.6 固化框架内部的实现逻辑

目前框架内部的功能实现仅作为占位符保证了的可用性，稳定性比起成熟的工具还有点欠缺。为保证后期框架的稳定性与可维护性，需要将各个模块的内部实现逻辑替换为主流成熟的工具。